MINIMA MORE

ENGINEERING PERFORMANCE BRIEF

THE NEW OPERATING ENVELOPE

Native C++ optimisation of the Minima blockchain node

Measured against the Java reference on ARM hardware and live mainnet

RESEARCH SNAPSHOT / 19 OCT 2026

01 / OUTCOMES

A smaller node. A wider deployment surface.

Four headline results; mainnet resource results and isolated cryptographic measurements are deliberately separated.

~85%

LESS IDLE RAM

Live mainnet on OCI A1
210 MB vs 1,392 MB

~75%

SMALLER DATA DIR

Live mainnet same-host snapshot
220 MB vs 881 MB

3.3×

SHA3-256 RATE

Single-thread 224-byte hashes
1.16M vs 0.346M hash/s

4.7×

WOTS SIGNING

Batched x4 / bounded workers
0.92 ms vs 4.30 ms

THE ENGINEERING THESIS

Preserve the consensus protocol while removing runtime, allocation, indexing and I/O overhead.

02 / THE OPPORTUNITY

Full-node sovereignty should not require server-class overhead.

The practical constraint is not one slow function; it is cumulative cost across the complete node lifecycle.

JAVA REFERENCE

302 MB

Fresh-chain idle process RSS

52 OS threads

THE OPTIMISATION TARGETS

MEMORY	MMR object graph / heap allocations
CPU	SHA3, WOTS and repeat recomputation
STORAGE	SQLite commits, reads and filesystem churn
NETWORK	Peer discovery, compatibility and sync

Small-device suitability depends on actual synced RAM, steady-state CPU, flash writes and successful mainnet operation.

03 / IMPLEMENTATION

Six optimisation surfaces. One coherent node.

The work spans protocol ingress, validation, in-memory state, execution and persistence.

NETWORK

P2P + IBD

Framing fix, bootstrap seeds, peer persistence

CRYPTO

WOTS + SHA3

Batched signatures, thread-local EVP

LOOKUP

MEMPOOL

Coin-to-TxPoW index, O(1) checks

STATE

MMR

Inlined entries, MiniBuffer SBO, root cache

EXECUTION

TXPOW

Parallel nonce mining, fast serialization

PERSISTENCE

SQLITE

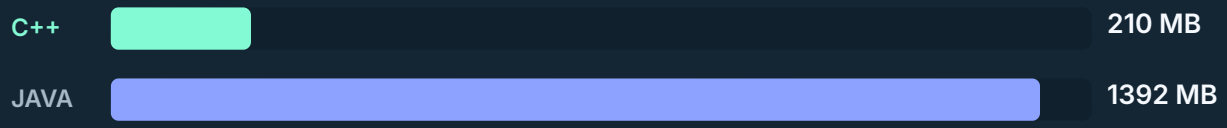
WAL tuning and batched transactions

04 / MAINNET MEASURED

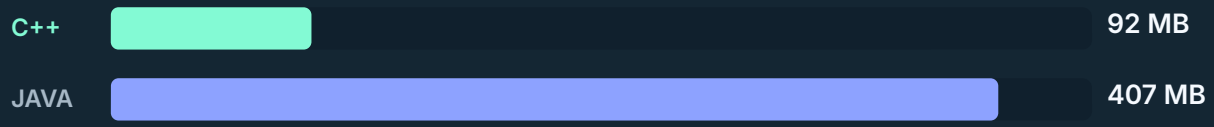
Live mainnet changes the RAM conversation.

Fresh C++ and Java nodes, same OCI Ampere A1 host, live peer sync; RAM shown as recorded process RSS.

Idle RSS average after initial sync



RSS snapshot labelled “peak during sync”



REPLICATED TREND

162 MB

C++ settled idle

705 MB

Java settled idle

Separate local-host run
19 Oct 2026

Interpretation: ~85% lower idle RAM on A1 (~77% lower on the local rerun). Sync and idle measurements use different windows.

05 / MAINNET MEASURED

Less CPU. Fewer threads. Smaller disk footprint.

Resource advantage is visible on the real network, but initial synchronisation speed is not yet a win.

CPU during settle

~49% less

C++ 34.3 s | Java 67.8 s

CPU-seconds / 60-second window

Operating threads

~58% fewer

C++ 25 | Java 60

threads after sync

Mainnet data dir

~75% smaller

C++ 220 MB | Java 881 MB

snapshot; divergent tip heights

SYNC SPEED TRADE-OFF

Time to tip: 10.0 s (C++) versus 6.2 s (Java) on this run.

06 / FUNCTIONAL VALIDATION

The node has crossed the real-network boundary.

Not just hashes per second: mainnet transactions and the protocol execution path were exercised end to end.

01

SYNC

Reached live tip and followed blocks

02

RECEIVE

Real incoming spend and confirmations

03

SEND

WOTS-sign, mine, broadcast; external confirmation

04

MINT

Token created and confirmed on mainnet

05

SCRIPT

KISSVM runscript executed

Peer discovery now reaches multiple live peers; ~10 reported in functional work, versus one before framing/bootstrap fixes.

07 / MEMORY ARCHITECTURE

The largest memory win was structural.

The MMR stopped paying for ~5-6 small heap allocations per entry and now keeps common 32-byte payloads inline.

BEFORE

Pointer-rich object graph



858 B / entry

AFTER

By-value objects + 32 B SBO

ENTRY | DATA | INLINE 32 B

634 B / entry

1M-entry MMR peak process RAM: 624 MB (C++) versus 1.90 GB (Java) on OCI A1 — ~67% less.

08 / COMPUTE PATHS

Removing repeated work compounds across the tree.

The optimisations target every hash, every MMR build and repeated root/peaks queries.

−41%

MMR 1M build time

3,785 → 2,216 ms (Apple M1)

~137×

MMR getRoot()

Mutation-invalidated cached root

~1,200×

MMR getPeaks()

Cached peaks vs recomputation

WHY IT WORKS

Direct-to-buffer crypto serialization + 64-bit number fast paths + cached MMR peaks/root.

Note: the MMR build result above is the project's on-M1 optimisation delta, not a claim of faster cross-host consensus.

09 / CRYPTOGRAPHY

Same SHA3 and WOTS. Faster execution.

Optimisation is in the implementation: OpenSSL EVP reuse, SIMD batching and bounded parallel work.

SHA3-256 / 224 B

3.3×

1.16M hashes/s C++

0.346M hashes/s Java

WOTS SIGN / BATCHED

4.7×

0.92 ms C++

4.30 ms Java

Inside C++ alone, the batched WOTS path was ~3.05× faster than its scalar path on OCI A1.

10 / ALGORITHMIC OPTIMISATION

An O(1) check replaced a full mempool scan.

A coin → TxPoW hash index accelerates duplicate-spend detection inside the transaction pool.

0.89 µs

C++ indexed coin lookup

43,733 µs

Java linear scan (~43.7 ms)

~49,000×

faster coin check

Scoped benchmark: 200k transactions, two coins each. Not a whole-node throughput multiplier.

11 / OPERATOR ECONOMICS

The lifecycle measurements reinforce the direction.

Same 4-core A1 host, scripted private-chain lifecycle; they are not equivalent to full historical mainnet sync.

~39%

LOWER IDLE CPU

38 vs 62 CPU-s / hour

~38%

FEWER DISK WRITES

3.7 vs 6.0 MB / hour

~31%

LOWER CPU / TX

1,635 vs 2,363 ms / tx

~77%

LESS SEND LATENCY

447 vs 1,965 ms / send

Idle CPU and disk-write rates were extrapolated from 180-second windows; transaction load is RPC-driven and run-dependent.

12 / STORAGE OPTIMISATION

Optimise SQLite today. Benchmark role-specific engines next.

The deployed tuning and the experimental storage-engine selection must not be conflated.

IMPLEMENTED

2.12×

Higher TxPoW insert throughput

1,519 → 3,222 rows/s on A1

WAL + device-aware tuning + 512-row batches

RESEARCH / NOT DEPLOYED

LMDB 1.75 µs

Coin point-get p50

DuckDB 2 ms

Archive analytics (SQLite: 23 ms)

M1 synthetic 200k-block / 120k-coin engine harness; role-based engine switch is a proposal, not current production architecture.

13 / TRUST AND SCOPE

Consensus preservation is a testable engineering constraint.

Protocol compatibility, mainnet operation and product-surface parity are separate questions.

VECTORS	12/12 golden consensus vectors; MMR / WOTS tests
REAL NETWORK	Live mainnet receive, send, token creation and script execution
P2P CORRECTNESS	SINGLE_PING framing fixed; bootstrap peers and chain-ID checks
DEFENCE	MiniData wire cap reduced from 512 MB to 16 MB

SCOPE MATTERS

The Java comparison is a performance baseline, not a byte-parity oracle.

Some optional Java-era surfaces, notably Maxima / MDS, are removed from this fork. Do not imply full feature parity.

14 / NEXT PROOF POINTS

Turn promising engineering results into decision-grade evidence.

The next step is a controlled validation programme, not a new set of bigger headline numbers.

01

REPEAT

Multi-run medians, variance, matched chain height, quieter host

02

EXTEND

Archive + MegaMMR import, query p50/p95, proof correctness

03

DEPLOY

Raspberry Pi 4/5 tests, sustained sync, CPU / RAM / flash wear

04

HARDEN

Soak, power loss, parser fuzzing, cross-implementation network tests

RFC-0002 (Archive / MegaMMR mainnet node-mode benchmarking) is still a draft plan.

MINIMA MORE

SAME CHAIN. NEW OPERATING ENVELOPE.

~85% less live-mainnet idle RAM on A1.

Faster cryptographic paths. Leaner state. Proven real-network transactions.

The remaining job: replicate and validate across realistic node modes and devices.

github.com/Totem-Edge/minima-more

APPENDIX / SOURCE NOTES

Evidence, definitions and caveats.

Reference points for technical due diligence and externally shared versions of this presentation.

MAINNET A1	reference-bench/results/a1-mainnet-bench/MAINNET_BENCH_SUMMARY.md
A1 MICRO / MMR	reference-bench/results/a1-full-20261018/SWEEP_SUMMARY.md
MAINNET FUNCTION	reference-bench/MAINNET_TESTS.md
LIFECYCLE	reference-bench/RESULTS_LOG.md + README.md
ENGINE RESEARCH	reference-bench/engines/README.md
STORAGE / MODES	RFC-0001 / RFC-0002 / RFC-0003

IMPORTANT COMPARABILITY NOTES

• Mainnet runs are single-run shared-host observations; the paired nodes reached different tip heights. • A1 mainnet RAM uses process RSS; earlier Java MMR scaling also reports JVM heap-in-use, which is not RSS. • Lifecycle hourly rates are extrapolations from brief windows. • Synthetic engine results are not in-node engine deployment. • “Peak sync” and “idle” are measured in different windows.